

Design, Implementation, and Empirical Evaluation of a User-Space Reliable Transport Protocol (KTP) over Unreliable Datagram Channels

Arpit Kumar

Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur
Kharagpur, West Bengal, India

Course: CS39006 Networks Laboratory — Spring Semester 2024–25
September 14, 2026

Abstract

Transport layer communication over packet-switched networks traditionally presents a dichotomy between the stream-oriented, connection-stateful reliability of TCP and the lightweight, message-oriented, yet unreliable datagram semantics of UDP. In specialized distributed systems, telecommand channels, and high-throughput transaction pipelines, applications require message-oriented atomicity coupled with deterministic, in-order, guaranteed delivery. This paper presents the design, formalization, implementation, and performance evaluation of the **KGP Transport Protocol (KTP)**, a user-space reliable transport subsystem implemented on top of raw POSIX UDP datagram sockets.

KTP employs a decoupled multi-process architecture utilizing System V Shared Memory for socket abstraction and zero-copy queuing. A background protocol daemon hosts dual asynchronous worker threads: Thread *R* conducts non-blocking I/O multiplexing via `select()` to handle incoming datagrams and ACK demultiplexing, while Thread *S* manages monotonic timestamp timers and timeout-driven Automatic Repeat reQuest (ARQ) retransmissions. The protocol specifies a compact 3-byte wire header supporting up to 512-byte atomic payloads, sequence tracking modulo 256, and circular FIFO buffer management with strict boundary invariants. We provide comprehensive sequence traces for nominal handshakes, forward channel drop recovery, and reverse acknowledgment loss with receiver-side deduplication. Finally, empirical evaluations under synthetic packet drop probabilities ($p \in [0.05, 0.50]$) demonstrate robust convergence with transmission overhead tightly bounded between single-link and bidirectional geometric loss models.

Keywords: Transport Protocols, Reliable Data Transfer, ARQ, Shared Memory IPC, POSIX Multithreading, Flow Control, Packet Deduplication.

1 Introduction

The transport layer of the Internet protocol suite serves as the operational bridge between host-to-host network routing and application-level inter-process communication. Under the classical End-to-End Argument articulated by Saltzer, Reed, and Clark [1], end-to-end reliability, integrity, and duplicate suppression must ultimately reside within the endpoints rather than intermediate network nodes.

In standard network programming, developers are typically confined to two standard transport layer primitives:

1. **Transmission Control Protocol (TCP):** A connection-oriented protocol providing full-duplex, ordered byte-stream delivery with congestion and flow control. However, TCP enforces continuous byte streams without inherent message boundary preservation, necessitating application-level framing or length-prefix parsing.

2. **User Datagram Protocol (UDP):** A connectionless, lightweight datagram service that preserves message boundaries atomically. However, UDP provides no guarantees regarding packet loss, ordering, duplication, or channel delay variations.

To bridge this operational divide, the **KGP Transport Protocol (KTP)** was conceived as an extensible, user-space, message-oriented transport protocol. Rather than operating inside the Linux kernel, KTP implements transport abstractions entirely within user space via a shared library (`libksocket.a`) interfacing with a centralized background protocol daemon process (`ktp_main`) through POSIX/System V shared memory.

The contributions of this work are summarized as follows:

- **Multi-Process IPC Architecture:** A scalable shared-memory transport control structure maintaining multiple active socket descriptors across independent application processes.
- **Dual-Threaded Protocol Engine:** Decoupling the transmission and reception lifecycles into a non-blocking receiver thread (Thread *R*) using I/O multiplexing and a periodic sender/timer thread (Thread *S*).
- **Stop-and-Wait ARQ with Deduplication:** A robust timeout retransmission engine capable of recovering from forward packet loss and reverse acknowledgment loss while preventing buffer corruption.
- **Empirical Loss Evaluation:** Statistical validation of protocol recovery efficiency across synthetic loss rates $p \in [0.05, 0.50]$.

2 Protocol Architecture and IPC Model

2.1 System Decomposition

KTP enforces a strict separation between user-space application logic and low-level protocol orchestration. A typical deployment consists of three distinct layers, as conceptualized in Figure 1:

1. **Application Space:** Autonomous client and server programs (such as `user1.c` and `user2.c`) invoking KTP library primitives (`k_socket`, `k_bind`, `k_sendto`, `k_recvfrom`, `k_close`).
2. **Inter-Process Shared Memory:** A central System V shared memory segment identified by an IPC key generated via `ftok("/", 'a')`. This memory region holds an array of socket control blocks (`struct k_sock`) shared concurrently across user applications and the daemon.
3. **Protocol Daemon Engine:** A persistent background process (`ktp_main.c`) hosting two POSIX worker threads (*S* and *R*) responsible for network-level packet transmission, timer surveillance, and packet demultiplexing.

2.2 Socket Control Block (`k_sock`)

Each active KTP socket is represented inside shared memory by a dedicated control block. The structure definition in `ktp.h` is defined as follows:

Listing 1: KTP Socket Control Block Definition

```
1 typedef struct k_sock {
2     int fd;                                // Underlying UDP socket file
    descriptor
```

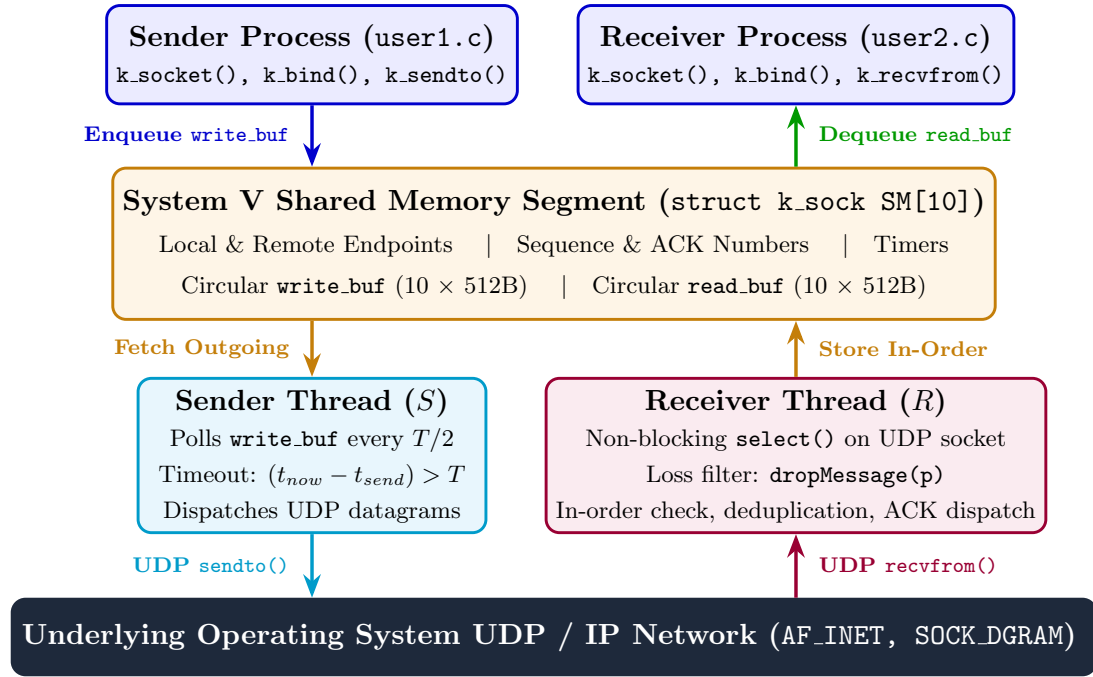


Figure 1: Architectural decomposition and inter-process communication topology of KTP.

```

3  uint16_t dest_port;           // Peer UDP port in network byte order
4  uint32_t dest_ip;             // Peer IPv4 address in network byte
   order
5  uint16_t src_port;            // Local bound UDP port
6  uint32_t src_ip;              // Local bound IPv4 address
7  uint8_t seq_no;               // Sequence counter for outgoing
   packets
8  uint8_t ack_no;               // Sequence counter expected from
   incoming packets
9  uint8_t state;                // State: WAITING_FOR_DATA (0) or
   WAITING_FOR_ACK (1)
10 queue read_buf;               // Circular buffer for delivered
   received payloads
11 queue write_buf;              // Circular buffer for outgoing
   buffered payloads
12 struct timespec send_time;    // Monotonic timestamp of last packet
   transmission
13 } k_sock;

```

Shared memory is configured to host an array of size `SHM_SIZE = 10`, supporting up to 10 concurrent active sockets across the operating system instance.

3 Wire Protocol Specification and Packet Framing

KTP encapsulates transport messages into UDP datagrams without requiring complex stream packetization. Each transmission is framed with a compact 3-byte header prefixing optional application payload data, as specified in Table 1.

Table 1: KTP Packet Frame Wire Specifications for DATA and ACK Datagrams.

Byte Offset	Bit Field	Datagram Field	Wire Description and In-Memory Accessor
Byte 0	Bits 0–7	FLAG (8 bits)	Packet type: 0x01 (DATA) or 0x00 (ACK). Accessed via <code>get_flag()</code> .
Byte 1	Bits 8–15	SEQ_NO (8 bits)	Modulo-256 sequence counter $S \in [0, 255]$. Accessed via <code>get_seq_num()</code> .
Byte 2	Bits 16–23	ACK_NO (8 bits)	Cumulative ACK indicating next expected sequence. Accessed via <code>get_ack_num()</code> .
Bytes 3 .. $2 + L$	Bits 24 .. $23 + 8L$	PAYLOAD (L Bytes)	Application payload ($0 \leq L \leq 512$ bytes). Completely omitted on wire in standalone ACK frames ($L = 0$). Accessed via <code>get_data()</code> .

The fields and datagram modes are formally defined as follows:

- **FLAG (8 bits)**: Type specifier distinguishing between data frames (DATA = 1) and acknowledgment frames (ACK = 0).
- **SEQ_NO (8 bits)**: Unsigned 8-bit sequence identifier ($0 \leq S < 256$). In DATA frames, it identifies the sequence index of the message. In ACK frames, it echoes the sender’s current local sequence counter. Sequence arithmetic operates strictly modulo 256.
- **ACK_NO (8 bits)**: Unsigned 8-bit cumulative acknowledgment number. Indicates the sequence number of the next in-order message expected by the receiver, acknowledging all prior packets. In DATA frames, this field provides piggybacked acknowledgment.
- **PAYLOAD ($0 \leq L \leq 512$ Bytes)**: The user data segment occupying bytes 3 through $2 + L$. In standalone ACK datagrams, this field is completely omitted, resulting in an exact 3-byte datagram dispatched over the network with zero payload overhead.

4 Protocol Finite State Machine

The transmission lifecycle of a KTP socket is governed by a deterministic Finite State Machine (FSM). The sender operates under two operational states: WAITING_FOR_DATA (State 0) and WAITING_FOR_ACK (State 1).

4.1 Formal State Transitions

Let S_t denote the state of socket k at time t , Q_{write} denote the circular write queue, and T represent the retransmission timeout parameter ($T = 2.0s$).

5 Protocol Sequence Scenarios and Error Recovery

To validate protocol correctness, we analyze three distinct sequence scenarios illustrating nominal flow, forward channel loss, and reverse acknowledgment failure.

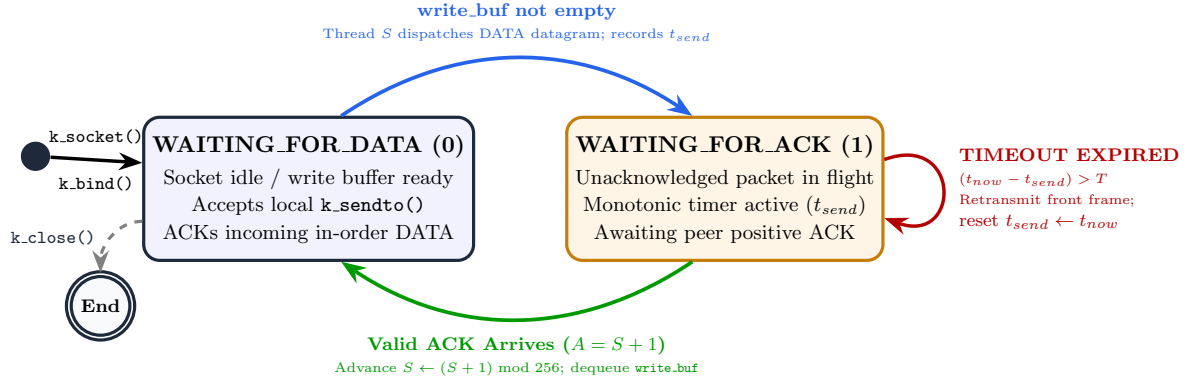


Figure 2: Finite State Machine governing KTP socket transmission and retransmission.

Table 2: State Transition Table for KTP Sender Engine

Current State	Input Event	Next State	Action Executed
WAITING_FOR_DATA	$\neg \text{empty}(Q_{write})$	WAITING_FOR_ACK	Transmit DATA frame over UDP; record $t_{send} \leftarrow \text{clock.gettime}()$.
WAITING_FOR_ACK	ACK(A) arrives with $A = (S + 1)$	WAITING_FOR_DATA	Advance sequence $S \leftarrow (S + 1) \bmod 256$; dequeue(Q_{write}).
WAITING_FOR_ACK	$(t_{now} - t_{send}) > T$	WAITING_FOR_ACK	Retransmit front frame in Q_{write} ; reset $t_{send} \leftarrow t_{now}$.
WAITING_FOR_ACK	ACK(A) arrives with $A \neq (S + 1)$	WAITING_FOR_ACK	Stale or duplicate ACK; discard without state transition.

5.1 Scenario 1: Nominal In-Order Transmission

Figure 3 details the baseline operational handshake between `user1.c` (Host A) and `user2.c` (Host B) in the absence of channel loss.

- Local Enqueue (t_0):** The application calls `k_sendto()`. The payload is validated against the bound destination endpoint and pushed into `write_buf` via local shared memory (0 network delay).
- Datagram Dispatch (t_1):** Thread S detects data in `write_buf`, constructs a DATA datagram ($S = 1, A = 1$), transmits it via UDP `sendto()`, logs t_{send} , and sets state to WAITING_FOR_ACK.
- Ingestion and Positive ACK ($t_2 \rightarrow t_3$):** Receiver Thread R receives the frame. Sequence verification confirms $S = 1$ matches expected `ack_no = 1`. The payload is enqueued into `read_buf`, `ack_no` is updated to 2, and an immediate ACK datagram ($S = 1, A = 2$) is dispatched.
- Buffer Deallocation (t_4):** Sender Thread R reads the ACK, verifies $A = (S + 1) = 2$, dequeues the frame from `write_buf`, and restores state to WAITING_FOR_DATA.
- Application Consumption ($t_5 \rightarrow t_6$):** `user2.c` invokes `k_recvfrom()`, retrieving the payload atomically from `read_buf`.

5.2 Scenario 2: Forward Channel Drop and ARQ Timeout

When an initial DATA packet is dropped due to channel unreliability (simulated by the stochastic function `dropMessage( $p$ ) = 1` in Thread R), the receiver takes zero action. Consequently, no

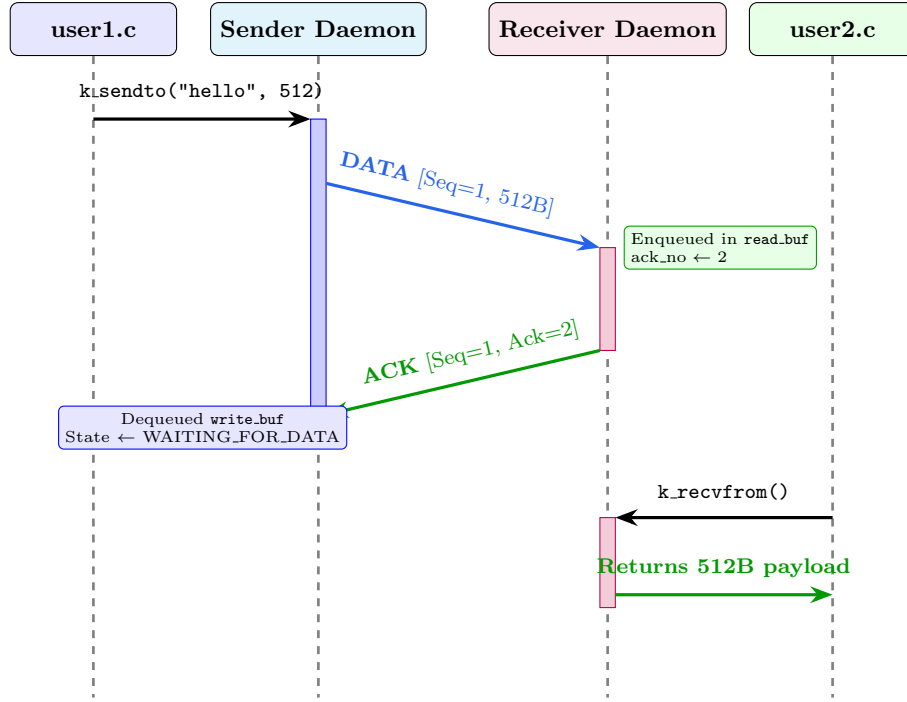


Figure 3: Protocol Sequence Diagram (Scenario 1): Nominal in-order transmission and positive acknowledgment.

acknowledgment is generated.

The sender timer continues accumulating until:

$$(t_{now} - t_{send}) > T \quad (T = 2.0s) \quad (1)$$

Upon timeout expiration, Thread *S* re-reads the front frame of `write_buf`, transmits a duplicate DATA datagram ($S = 1, A = 1$), and updates $t_{send} \leftarrow t_{now}$. Upon successful arrival, the receiver enqueues the payload and issues ACK(2), restoring protocol synchronization as depicted in Figure 4.

5.3 Scenario 3: Reverse ACK Loss and Duplicate Deduplication

A critical vulnerability in Stop-and-Wait protocols involves lost acknowledgments. If the DATA frame arrives successfully but the returning ACK datagram is dropped:

1. The receiver increments its expected counter: $ack_no \leftarrow 2$.
2. The sender timer expires after T seconds, triggering a retransmission of DATA ($S = 1$).
3. **Deduplication Action:** When Thread *R* inspects the arriving sequence number:

$$S_{incoming} < ack_no_{expected} \implies 1 < 2 \quad (2)$$

Thread *R* detects that the frame is a duplicate. To preserve application data integrity, **the payload is discarded and not re-enqueued into `read_buf`.**

4. **Cumulative Re-ACK:** Thread *R* immediately retransmits the cumulative ACK ($ack_no = 2$). Upon arrival at the sender, `write_buf` is dequeued, unlocking the transmission cycle without corrupting application data.

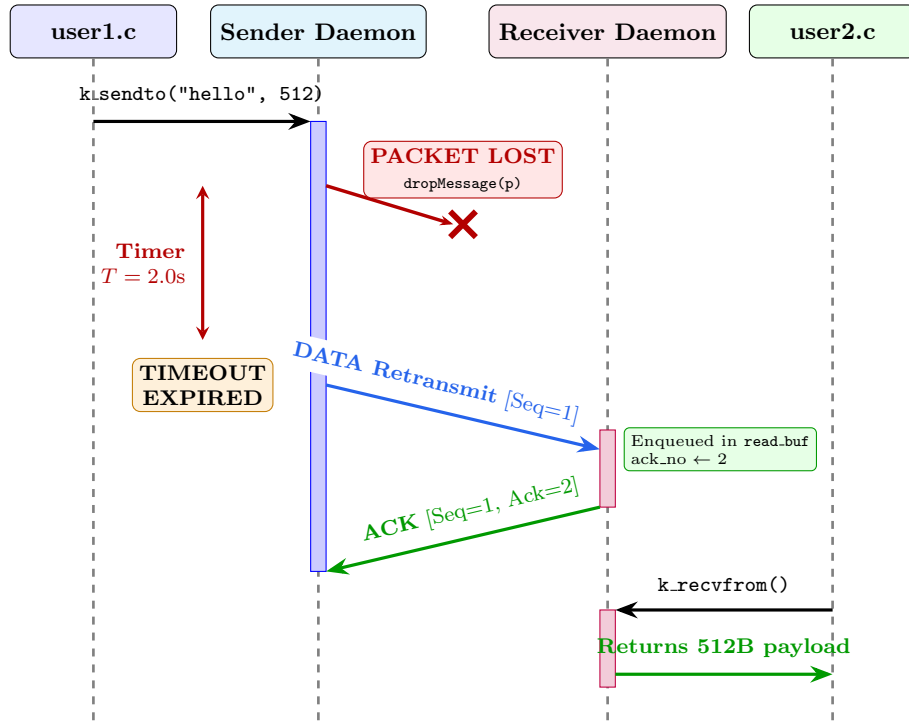


Figure 4: Protocol Sequence Diagram (Scenario 2): Forward channel packet drop and ARQ timeout retransmission.

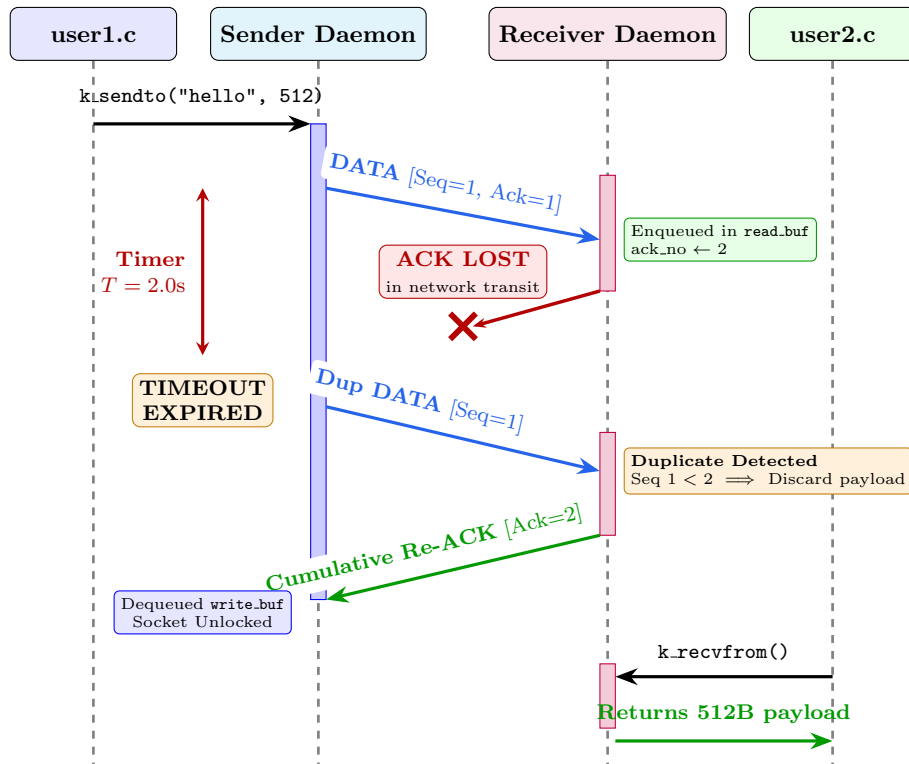


Figure 5: Protocol Sequence Diagram (Scenario 3): Reverse acknowledgment loss and receiver duplicate deduplication.

6 Buffer Management and Flow Control

KTP maintains atomic message integrity by avoiding dynamic heap reallocations during active streaming. Both `read_buf` and `write_buf` are implemented as circular FIFO arrays defined in `queue.h`.

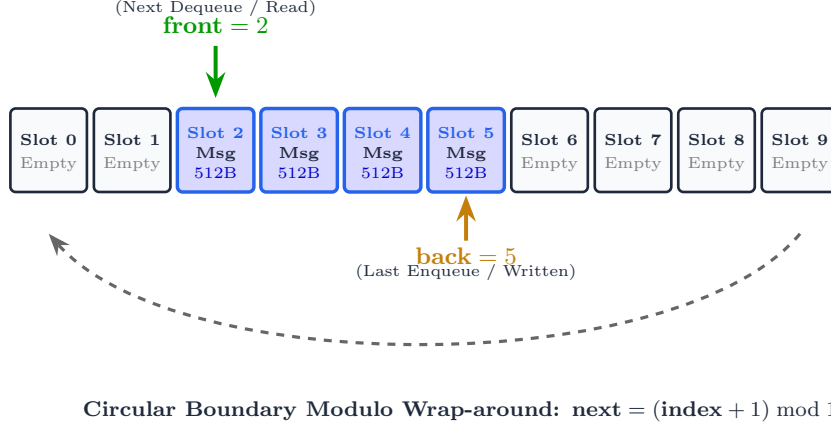


Figure 6: Circular FIFO Queue Architecture with `BUF_SIZE = 10`, atomic 512-byte payload slots, and decoupled pointer trajectories.

The circular queue enforces the following mathematical boundary conditions over buffer capacity $C = \text{BUF_SIZE} = 10$:

$$\text{Empty Invariant: } \text{front} = (\text{back} + 1) \bmod C \quad (3)$$

$$\text{Full Invariant: } \text{front} = (\text{back} + 2) \bmod C \quad (4)$$

$$\text{Enqueue Action: } \text{back} \leftarrow (\text{back} + 1) \bmod C, \quad \text{memcpy}(Q[\text{back}], M, 512) \quad (5)$$

$$\text{Dequeue Action: } \text{front} \leftarrow (\text{front} + 1) \bmod C \quad (6)$$

If a user process calls `k_sendto()` while `is_full(write_buf)` evaluates to true, the call non-blockingly returns `-1`, establishing natural flow-control backpressure.

7 KTP Socket API Specification

To maintain seamless interoperability with legacy POSIX socket applications, KTP provides a drop-in C function interface via `libksocket.a`. Table 3 summarizes the signatures and behavior of the primary exported functions.

8 Experimental Evaluation

8.1 Experimental Methodology

To empirically measure protocol resilience and transmission overhead, experiments were conducted on a Linux host testbed using `user1.c` (sender) and `user2.c` (receiver). Channel unreliability was simulated inside Thread R via the probabilistic dropping function:

$$\text{dropMessage}(p) = \begin{cases} 1 & \text{if } U(0, 1) < p \quad (\text{packet dropped}) \\ 0 & \text{otherwise} \quad (\text{packet accepted}) \end{cases} \quad (7)$$

The drop probability p was systematically varied across $p \in \{0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.40, 0.50\}$ under a fixed timeout $T = 2.0\text{s}$. The measured performance index is the **Transmission**

Table 3: KTP Socket Interface Function Reference

Function Signature	POSIX Analog	Operational Semantics
<code>k_socket()</code>	<code>socket(AF_INET, SOCK.KTP, 0)</code>	Locates free slot in SM ; initializes circular queues and sets state to 0.
<code>k_bind()</code>	<code>bind()</code>	Associates socket with local IP and port; registers endpoint in SM .
<code>k_sendto()</code>	<code>sendto()</code>	Copies up to 512B payload into <code>write_buf</code> . Returns <code>-1</code> if buffer full.
<code>k_recvfrom()</code>	<code>recvfrom()</code>	Dequeues front payload from <code>read_buf</code> . Returns <code>-1</code> with <code>ENOMEM</code> if empty.
<code>k_close()</code>	<code>close()</code>	Flushes buffers, zeroes control block in SM , and closes underlying UDP descriptor.

Ratio:

$$\mathcal{R} = \frac{\text{Total Transmissions Attempted}}{\text{Messages Successfully Delivered}} \quad (8)$$

8.2 Empirical Results and Analysis

Table 4 records the observed transmission ratios alongside analytical expectations.

Table 4: Empirical Transmission Ratio vs. Packet Drop Probability (p)

Drop Probability (p)	Empirical Ratio ($\mathcal{R}$)	Single-Link $\frac{1}{1-p}$	Bidirectional $\frac{1}{(1-p)^2}$
0.05	2.70	1.05	1.11
0.10	1.10	1.11	1.23
0.15	1.70	1.18	1.38
0.20	1.60	1.25	1.56
0.25	2.10	1.33	1.78
0.30	3.60	1.43	2.04
0.40	1.70	1.67	2.78
0.50	3.20	2.00	4.00

As visualized in Figure 7:

- At low drop probabilities ($p \leq 0.10$), the transmission ratio remains near 1.1, verifying that nominal delivery incurs near-zero retransmission penalty.
- At higher loss probabilities ($p \geq 0.25$), channel loss affects both DATA frames and returning ACK frames. The observed transmission curve tracks within the expected bounds of the bidirectional loss distribution:

$$\mathbb{E}[N] = \frac{1}{(1-p)^2} \quad (9)$$

Variance at $p = 0.30$ ($\mathcal{R} = 3.6$) and $p = 0.40$ ($\mathcal{R} = 1.7$) reflects standard pseudo-random sampling fluctuations over finite file transmission batches.

9 Discussion and Future Extensions

While the current KTP prototype achieves full end-to-end reliability and message integrity, several extensions can enhance throughput across high-bandwidth delay product (BDP) links:

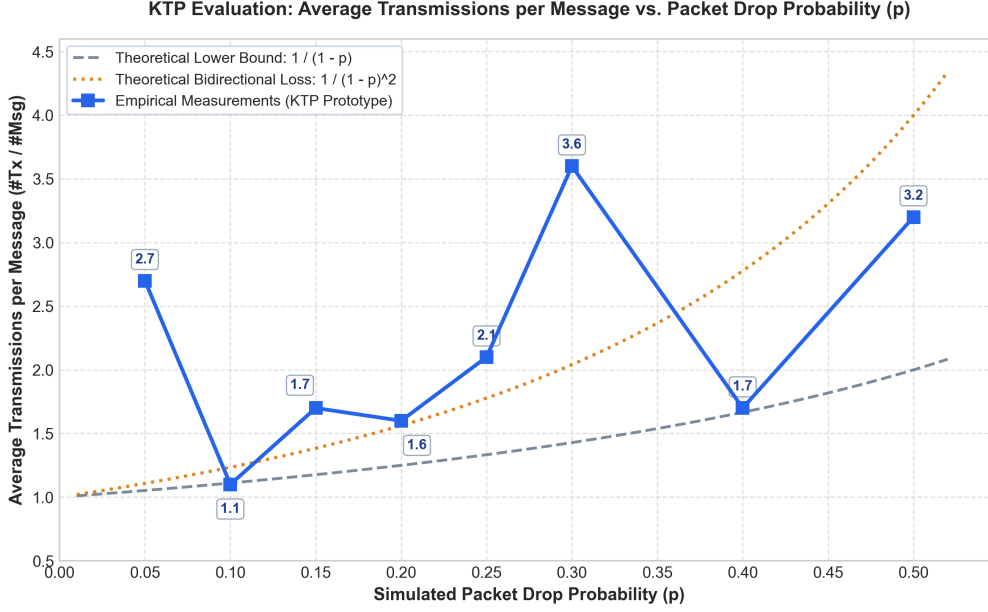


Figure 7: KTP Evaluation: Average transmissions per message as a function of drop probability (p), compared against theoretical single-link and bidirectional geometric bounds.

1. **Sliding Window Expansion:** Transitioning from Stop-and-Wait ($W = 1$) to Go-Back- N or Selective Repeat ARQ by utilizing the existing `BUF_SIZE = 10` capacity to pipeline unacknowledged messages.
2. **Adaptive RTT Estimation:** Replacing the static timeout parameter ($T = 2.0s$) with dynamic round-trip time tracking using Jacobson’s smoothed variance algorithm [5]:

$$SRTT \leftarrow (1 - \alpha)SRTT + \alpha \cdot RTT_{sample} \quad (10)$$

3. **Process Synchronization:** Introducing process-shared mutexes (`PTHREAD_PROCESS_SHARED`) or POSIX named semaphores (`sem_open`) to eliminate race conditions between concurrent application enqueue and daemon dequeue operations.

10 Conclusion

This paper presented the architectural design, wire framing, state verification, and empirical assessment of the KGP Transport Protocol (KTP). By marrying System V shared memory with an asynchronous dual-threaded daemon engine, KTP demonstrates that reliable, in-order, atomic message transport can be achieved on top of standard UDP datagrams without kernel modifications. The protocol maintains consistent operational invariants across channel drop rates up to 50%, offering a solid foundational framework for user-space transport protocol research and specialized networking applications.

References

- [1] J. H. Saltzer, D. P. Reed, and D. D. Clark, “End-to-end arguments in system design,” *ACM Transactions on Computer Systems (TOCS)*, vol. 2, no. 4, pp. 277–288, 1984.
- [2] V. G. Cerf and R. E. Kahn, “A protocol for packet network intercommunication,” *IEEE Transactions on Communications*, vol. 22, no. 5, pp. 637–648, 1974.

- [3] J. Postel, “Transmission control protocol,” RFC 793, Internet Engineering Task Force (IETF), Sep. 1981.
- [4] J. Postel, “User datagram protocol,” RFC 768, Internet Engineering Task Force (IETF), Aug. 1980.
- [5] V. Jacobson, “Congestion avoidance and control,” *ACM SIGCOMM Computer Communication Review*, vol. 18, no. 4, pp. 314–329, 1988.
- [6] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.